OBDX Development Factory Screens

Development build · captured 1 November 2026 · 1440 × 1000

About this document

OBDX Development Factory is a web tool that turns a bank's business requirements (BRDs, FSDs, prompts) into screen designs and code for its existing Oracle Banking Digital Experience (OBDX 22.x and 25.x) codebase, using local or privately hosted AI models.

This document shows every screen of the current development build, captured from a test factory with two demo bank workspaces (Bank A on OBDX 22.x, Bank B on OBDX 25.x) and five sample BRDs. We want feedback on how to make it simpler, more practical and more convincing for bank development teams.

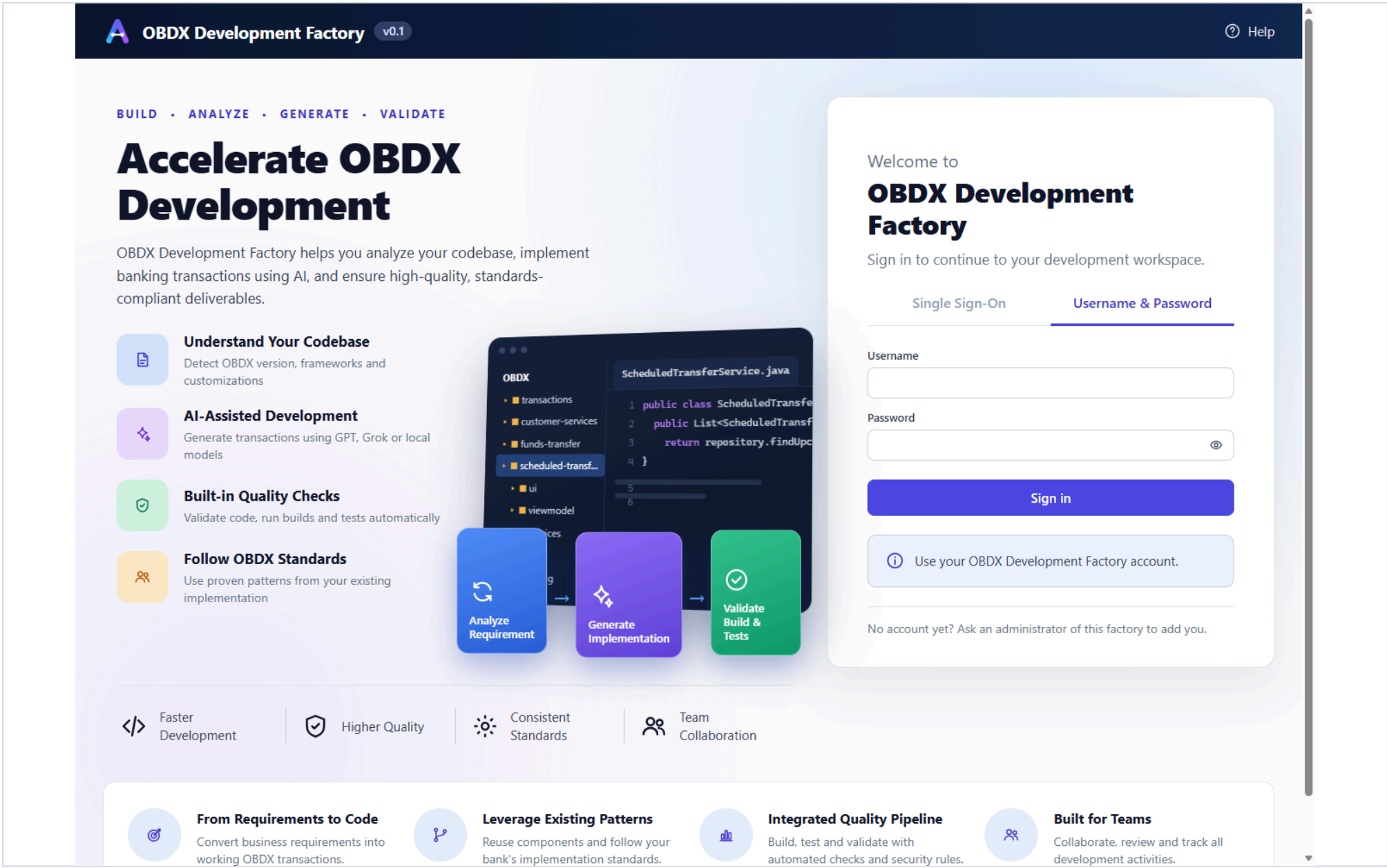
BRD	Kind of change	Workspace
BRD1 Standing Instructions	New transaction, end to end (UI, REST API, database)	Bank B, 25.x
BRD2 Transfer Funds Enhancement	Enhance an existing transaction	Bank B, 25.x
BRD3 Quick Transfer Widget	New transaction, UI and UX only	Bank B, 25.x
BRD4 Beneficiary Bulk Validation	New backend REST API only	Bank A, 22.x
BRD5 Transfer Funds for mobile corporate makers	Customise an existing transaction for one channel and user role	Bank B, 25.x

The flow from BRD to delivered code. A person approves the result of each AI step before the next one starts.

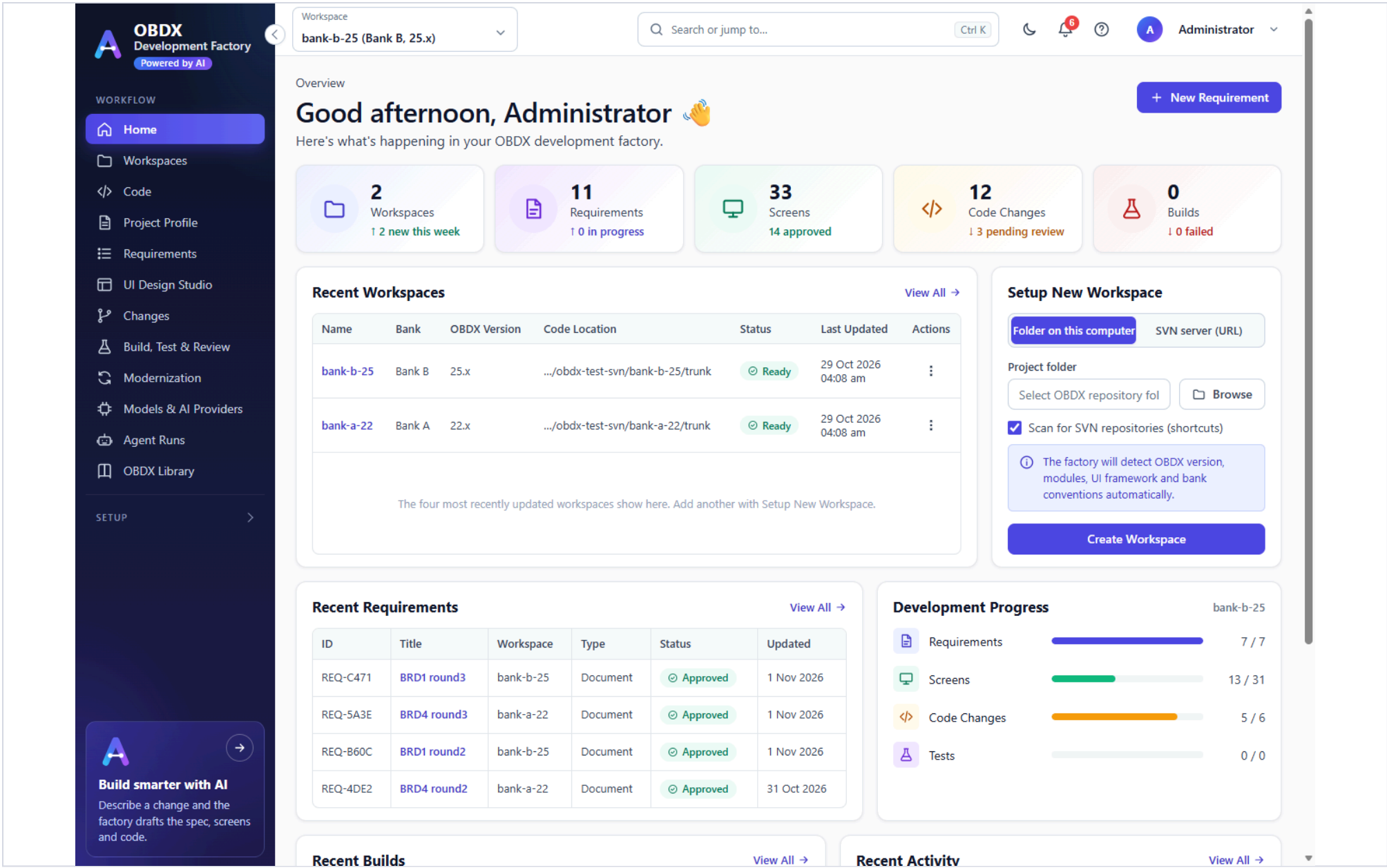
1. **Connect the code base.** A workspace points at the bank's OBDX source in SVN (or a folder). The factory imports and indexes it, and writes a project profile: OBDX version, modules, frameworks, customisations.
2. **Upload the BRD.** The factory reads the Word or PDF document, keeps its requirement IDs, scope and open questions word for word, and the model drafts a structured spec: requirements, acceptance criteria, screens, APIs, open questions.
3. **Answer and approve the spec.** The analyst answers the open questions (the model can suggest answers) and approves the spec.
4. **Design the screens.** The UI Design Studio draws each screen of each requirement in OBDX style, as a phone or desktop flow. The analyst refines them with AI or by hand and approves them.

5. **Plan the change.** Impact analysis lists every file to create or modify across UI, REST services, database and tests, with risks and a check of the plan against the spec.
6. **Generate and check the code.** The model writes each file; the factory checks it against the bank's patterns, security rules and the other files of the change, and sends problems back to the model.
7. **Review and deliver.** Developers review the diff, comment, build and test (Jenkins), and deliver to an SVN branch or download a patch.

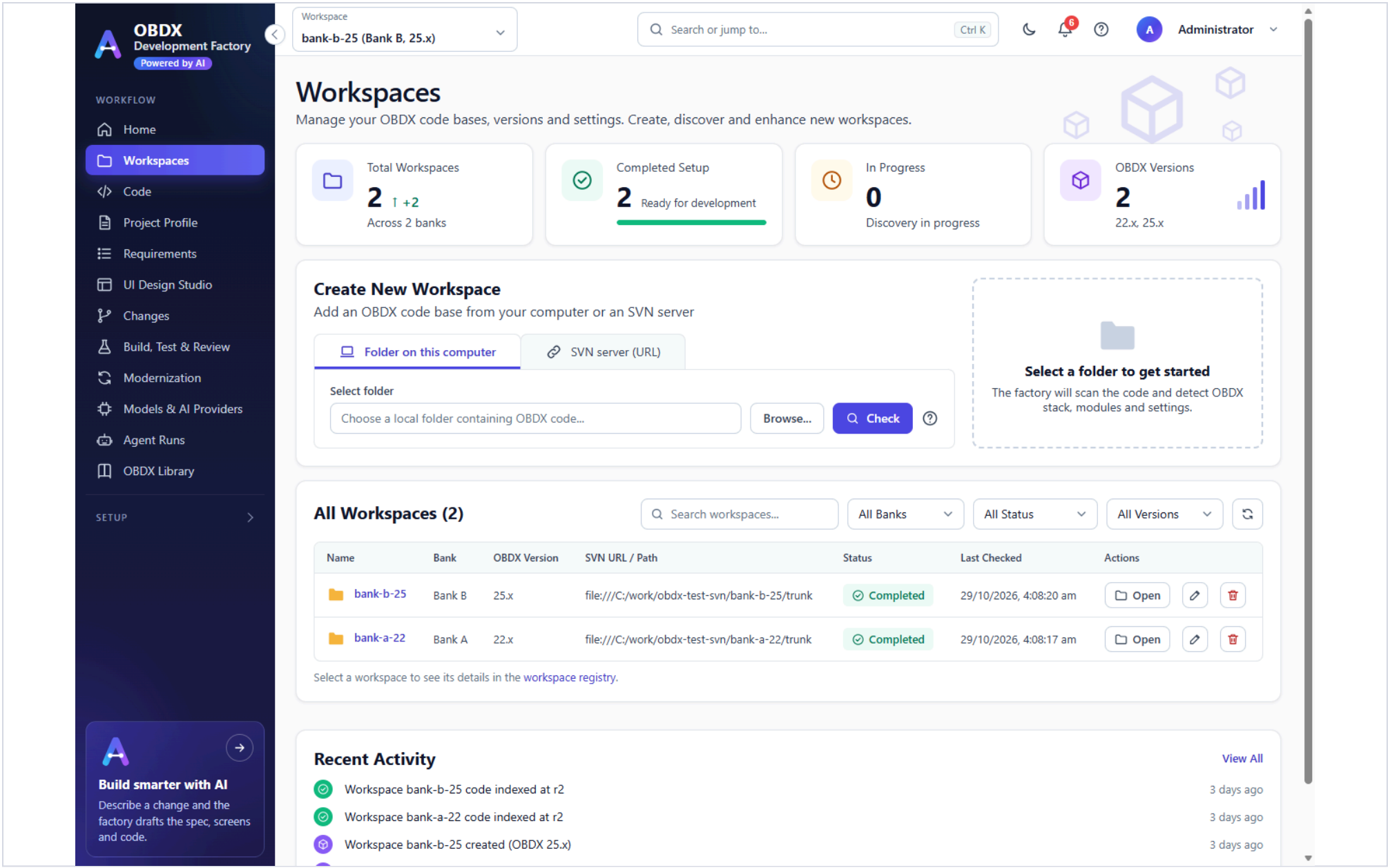
Getting started



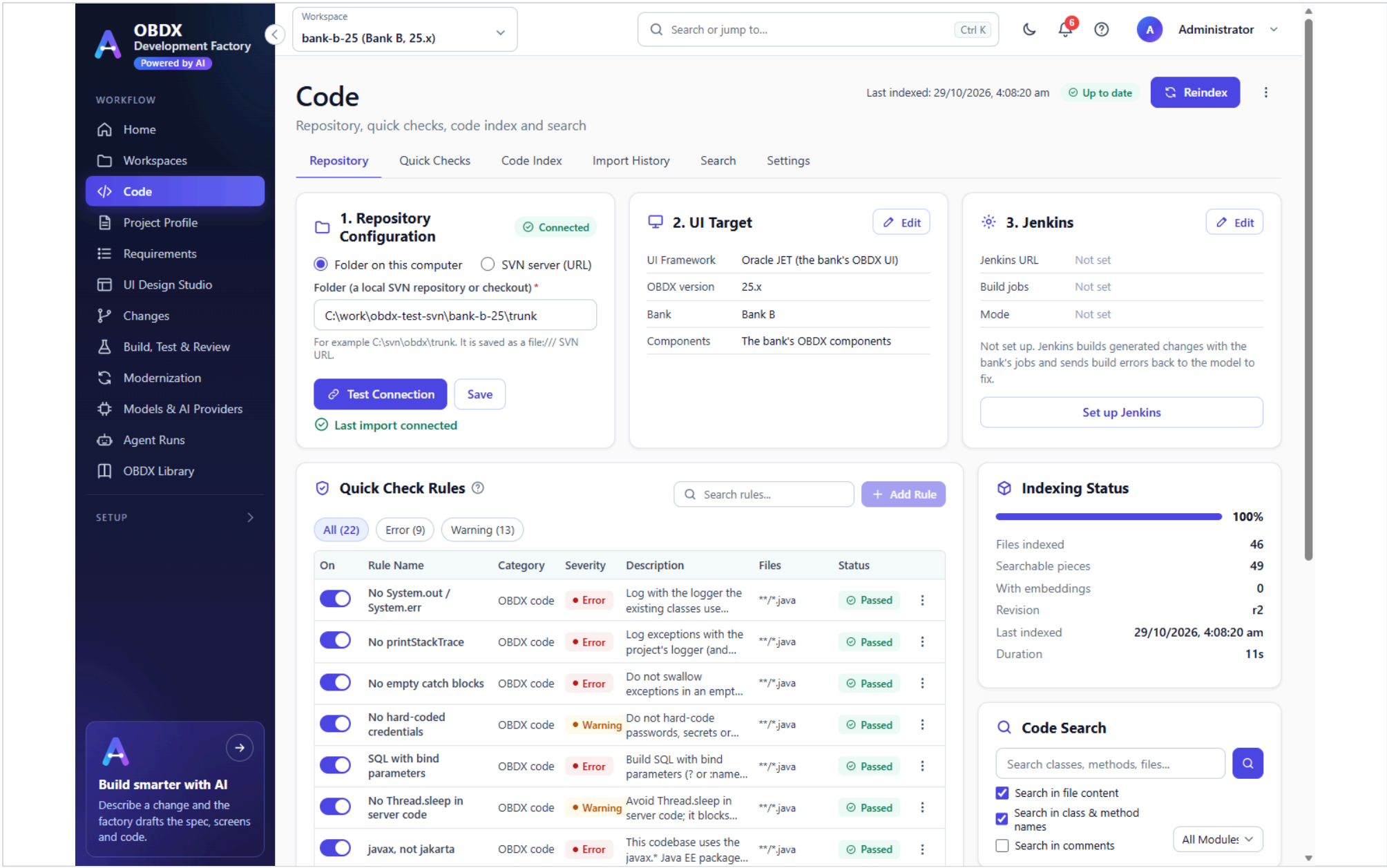
Sign-in. Local accounts, the bank's directory (LDAP) or single sign-on.



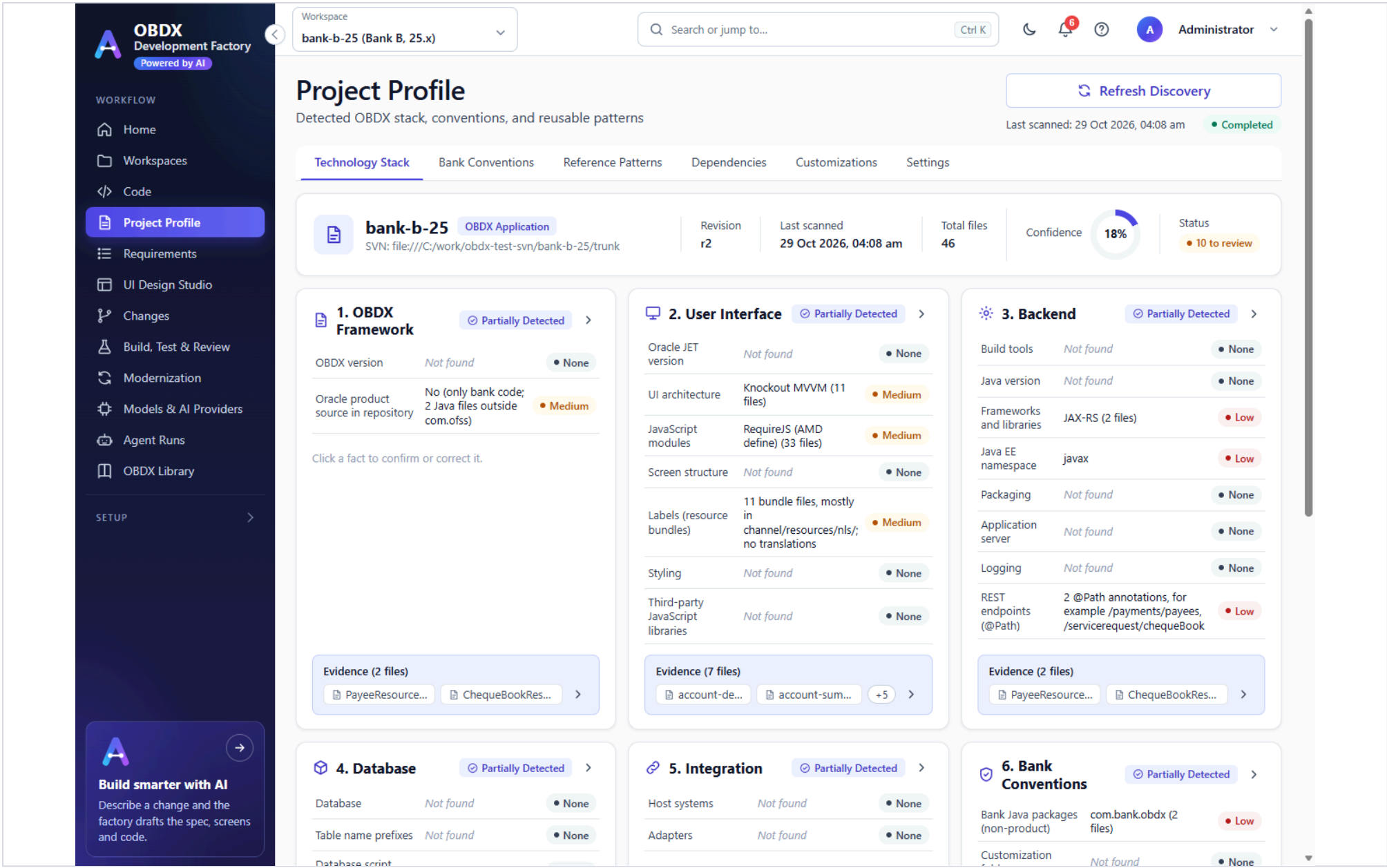
Home. The selected workspace's progress from requirements to delivered code, recent requirements and changes, and what needs attention.



Workspaces. One workspace per bank code base: its SVN source, OBDX version, import and indexing status.

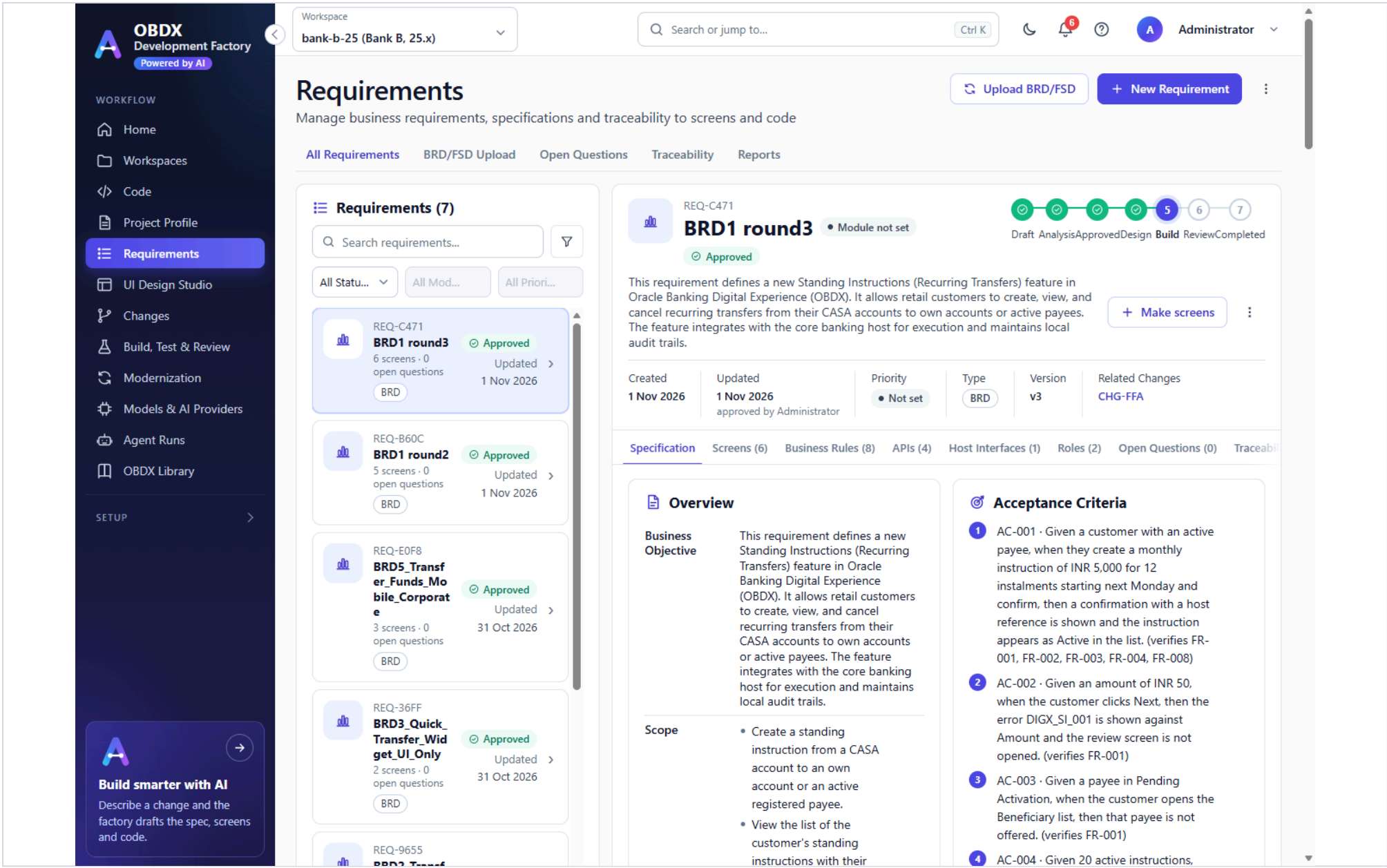


Code. Browse and search the imported code; the model's answers cite these files.

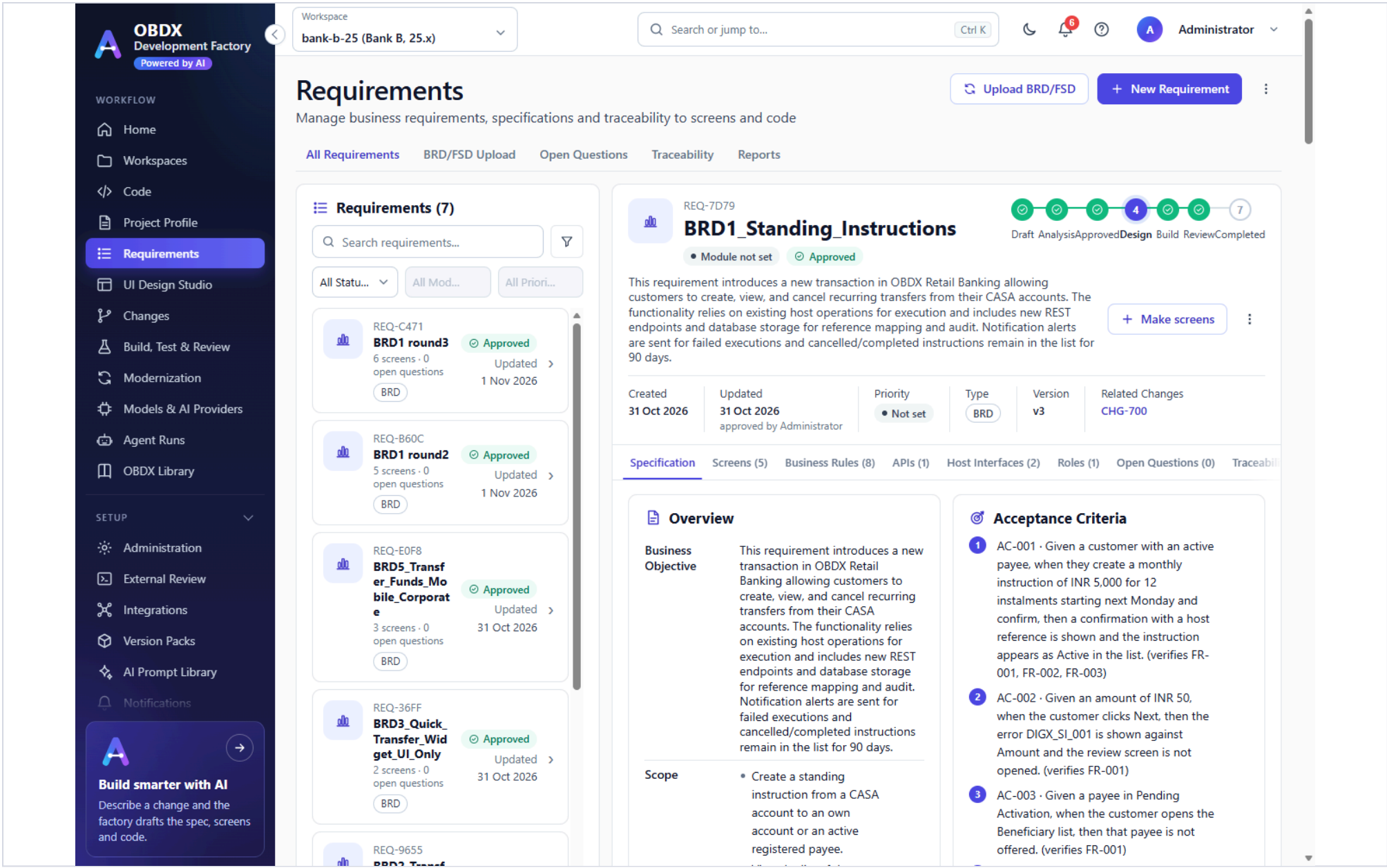


Project Profile. What the factory learned about the code base: OBDX version, modules, frameworks, customisation patterns. Generated code follows it.

Requirements



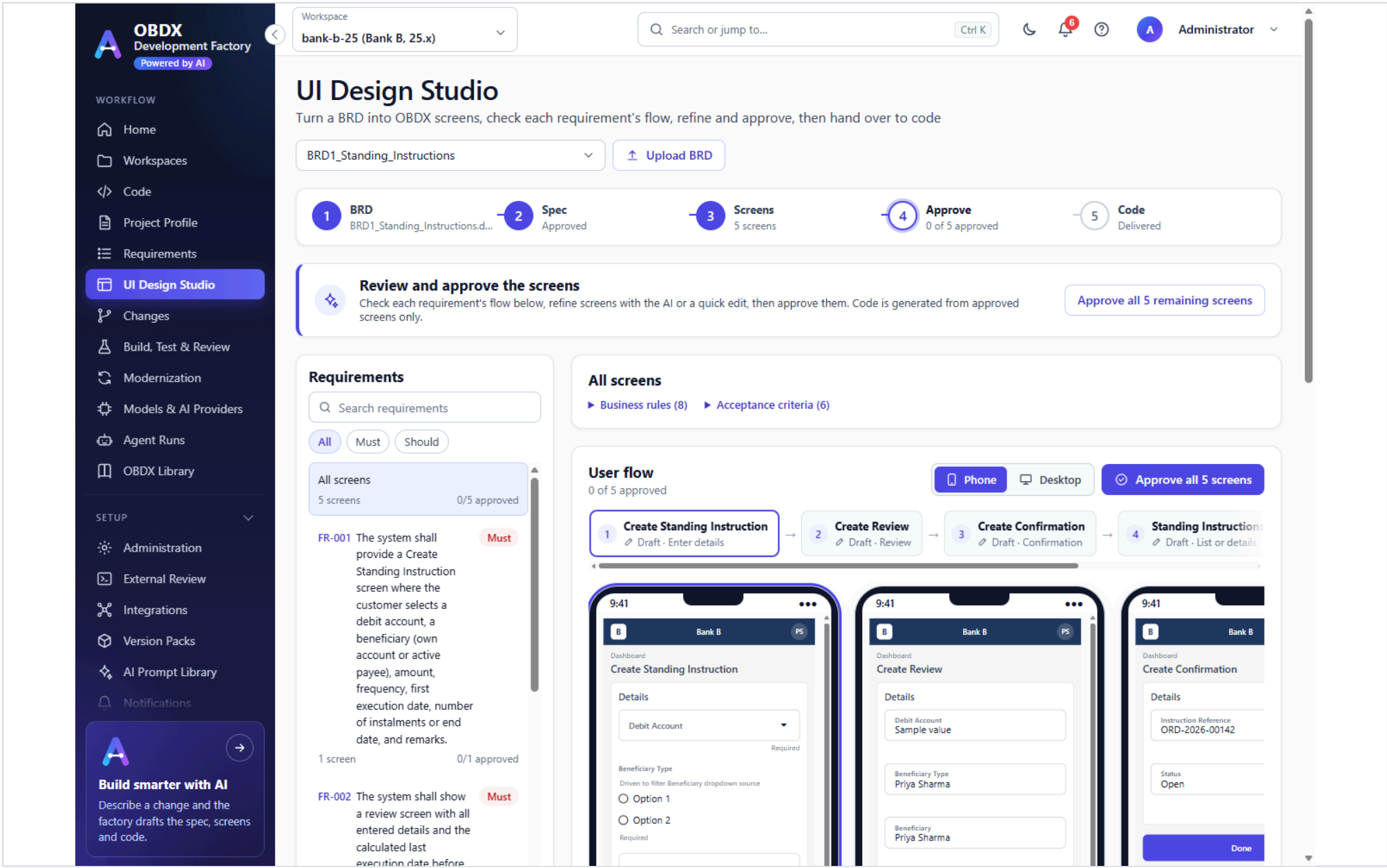
Requirements list. Every BRD, FSD or typed requirement of the workspace with its status, screens and open questions. Upload a BRD or FSD from here.



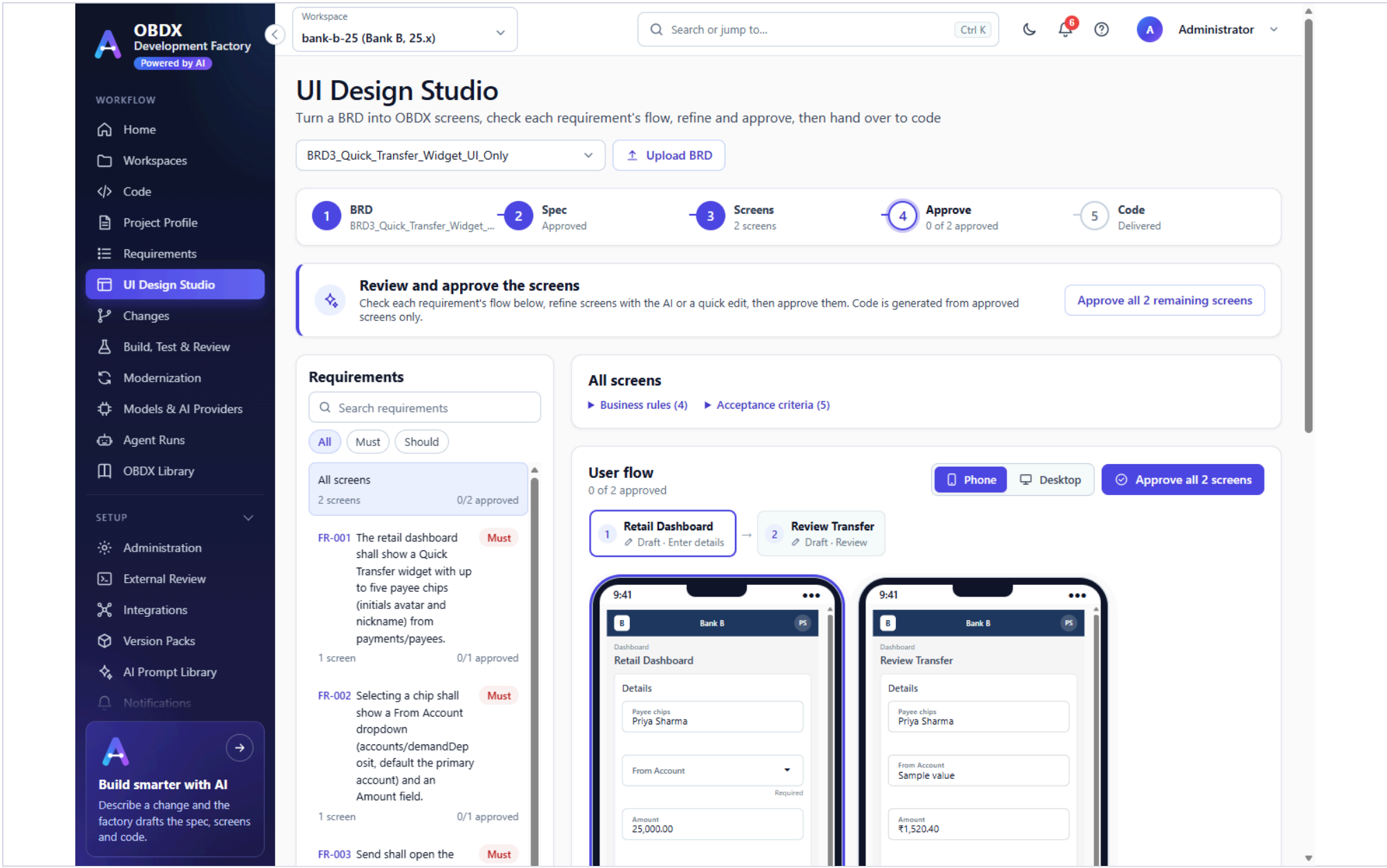
One requirement (BRD1 Standing Instructions). The drafted spec: business objective, scope, acceptance criteria with the requirements they verify, business rules, APIs, host interfaces, roles, open questions and traceability. Make screens sends it to the Design Studio.

UI Design Studio and Visual Designer

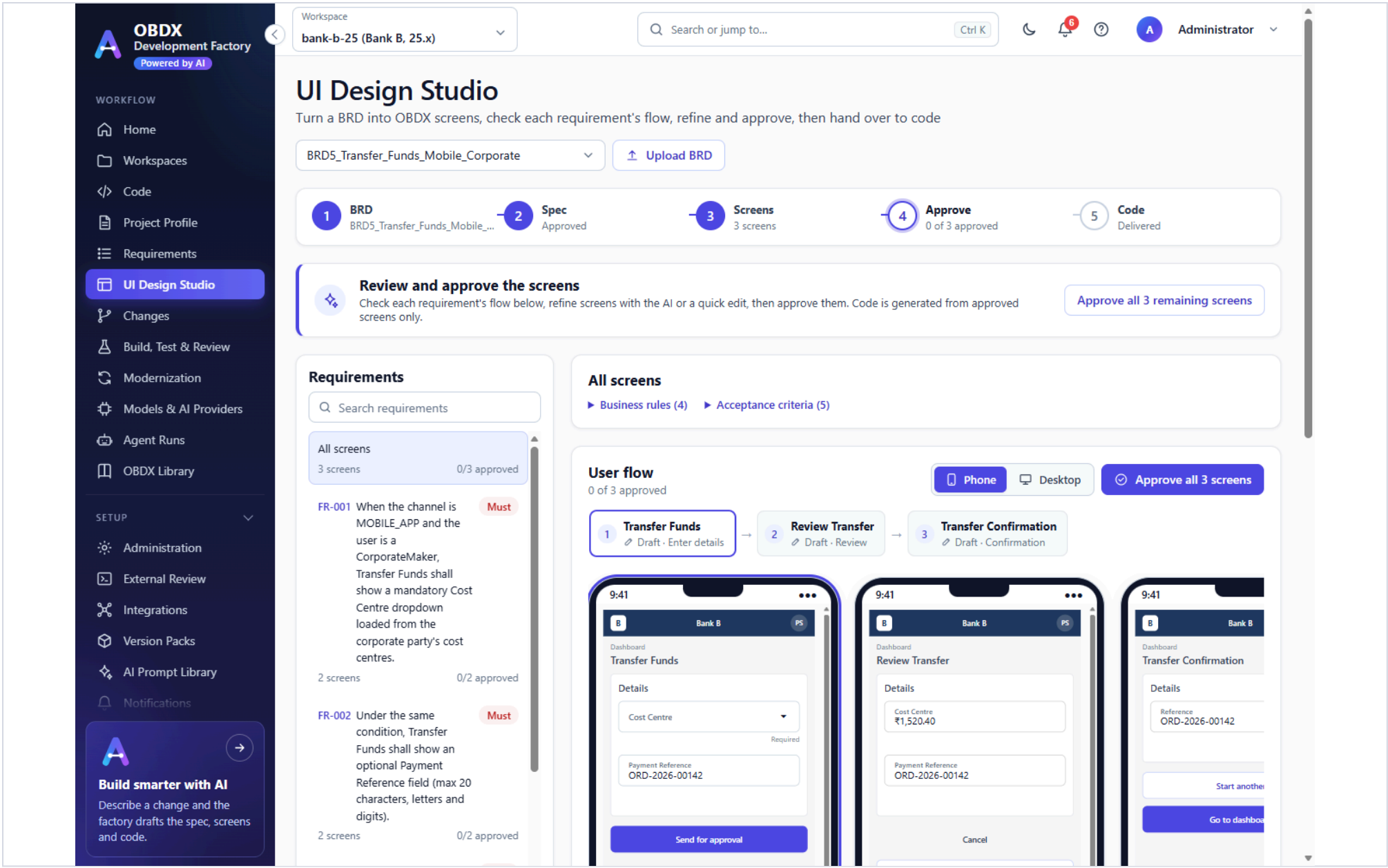
The Design Studio is the main selling point: one page takes a BRD to approved OBDX screens. A five-step bar shows BRD, spec, screens, approval and code; the left column lists the requirements with how many of their screens are approved; the right shows the selected requirement's user flow as phone or desktop previews in the bank's OBDX style.



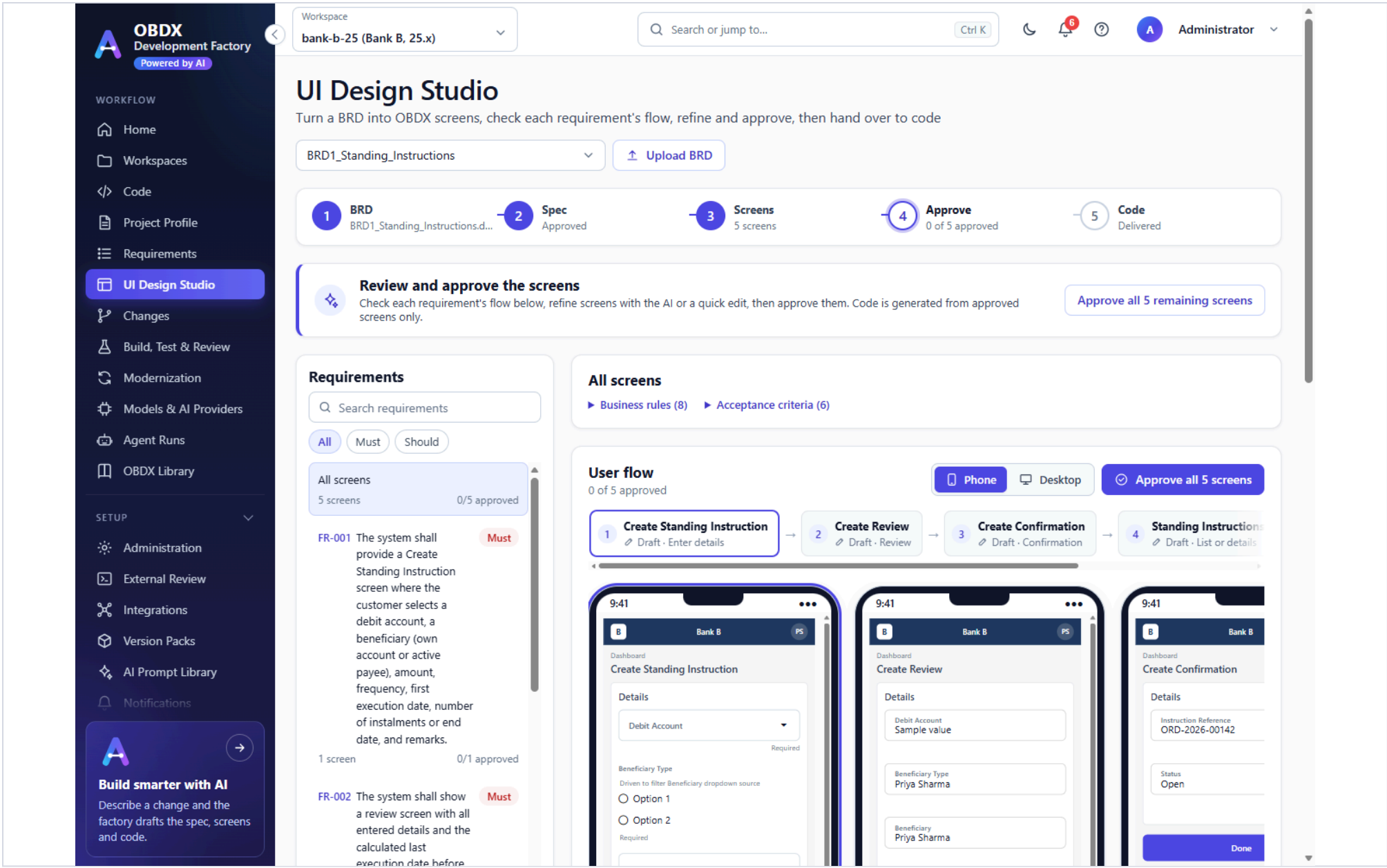
BRD1 Standing Instructions (new transaction, end to end). Five screens: create, review, confirmation, list and details.



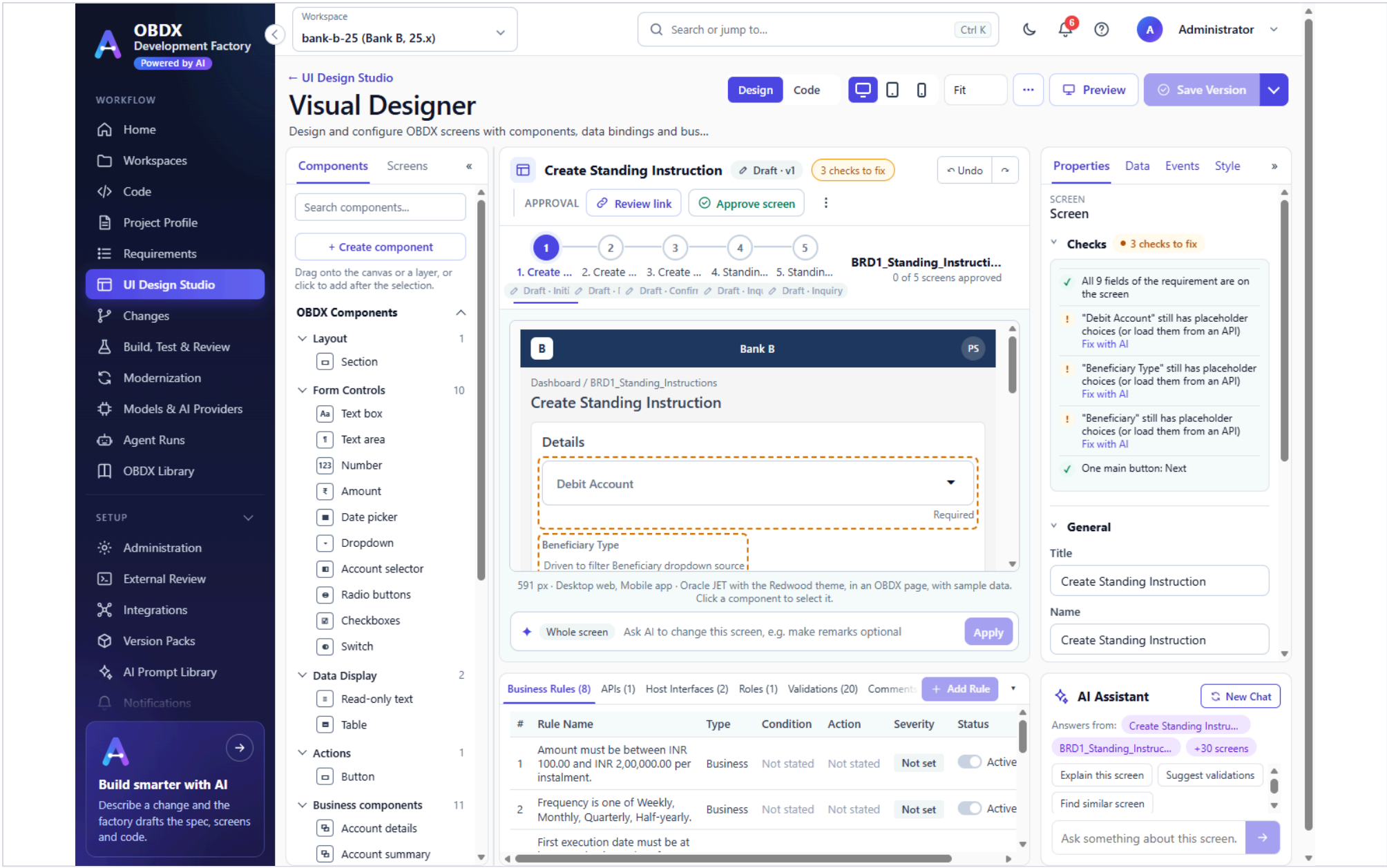
BRD3 Quick Transfer Widget (UI and UX only). A dashboard widget with payee chips and its review screen.



BRD5 Transfer Funds for mobile corporate makers (channel and role customisation). The existing transfer screens, mobile app only.

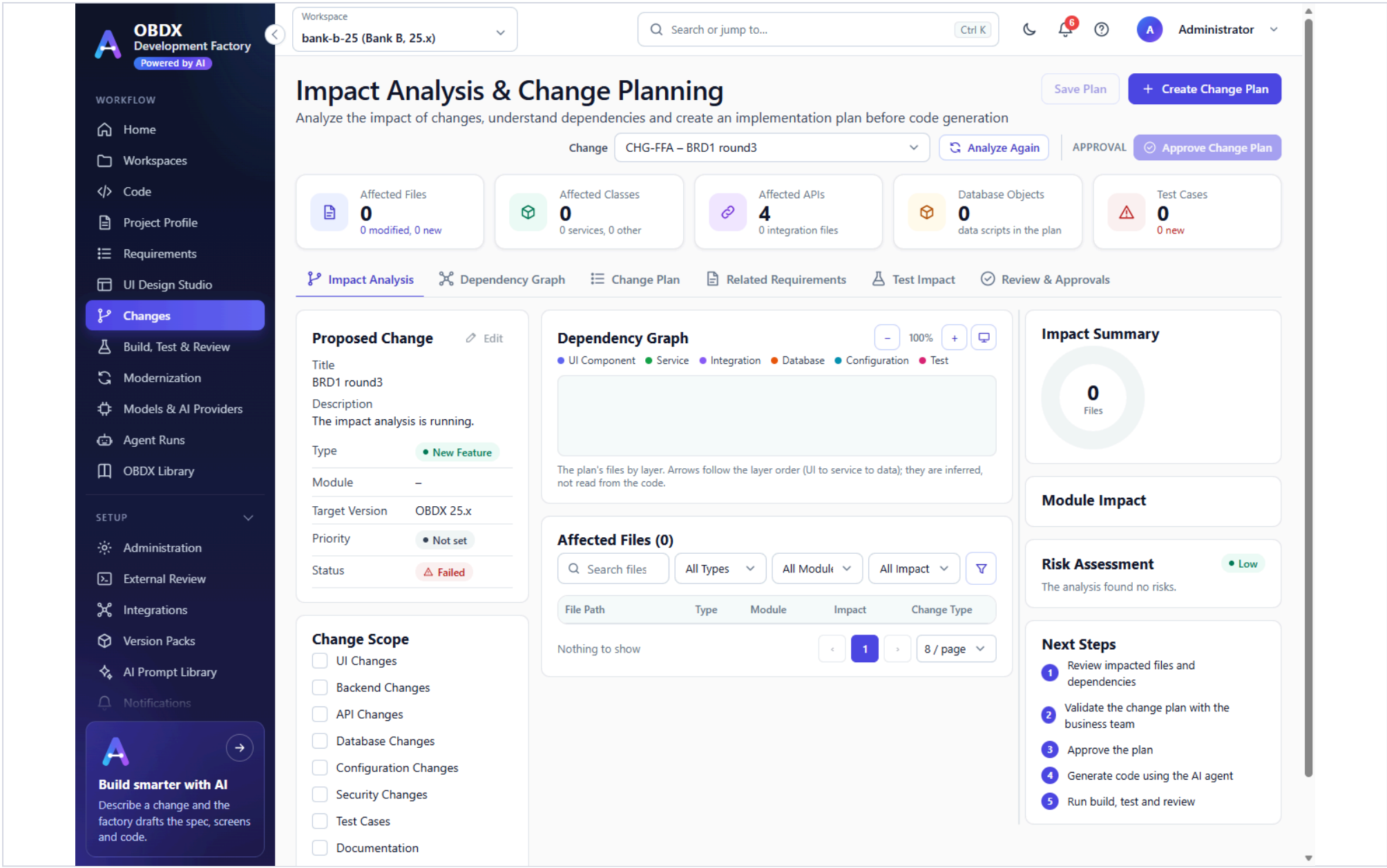


Dark theme. Every page follows the user's light or dark choice.

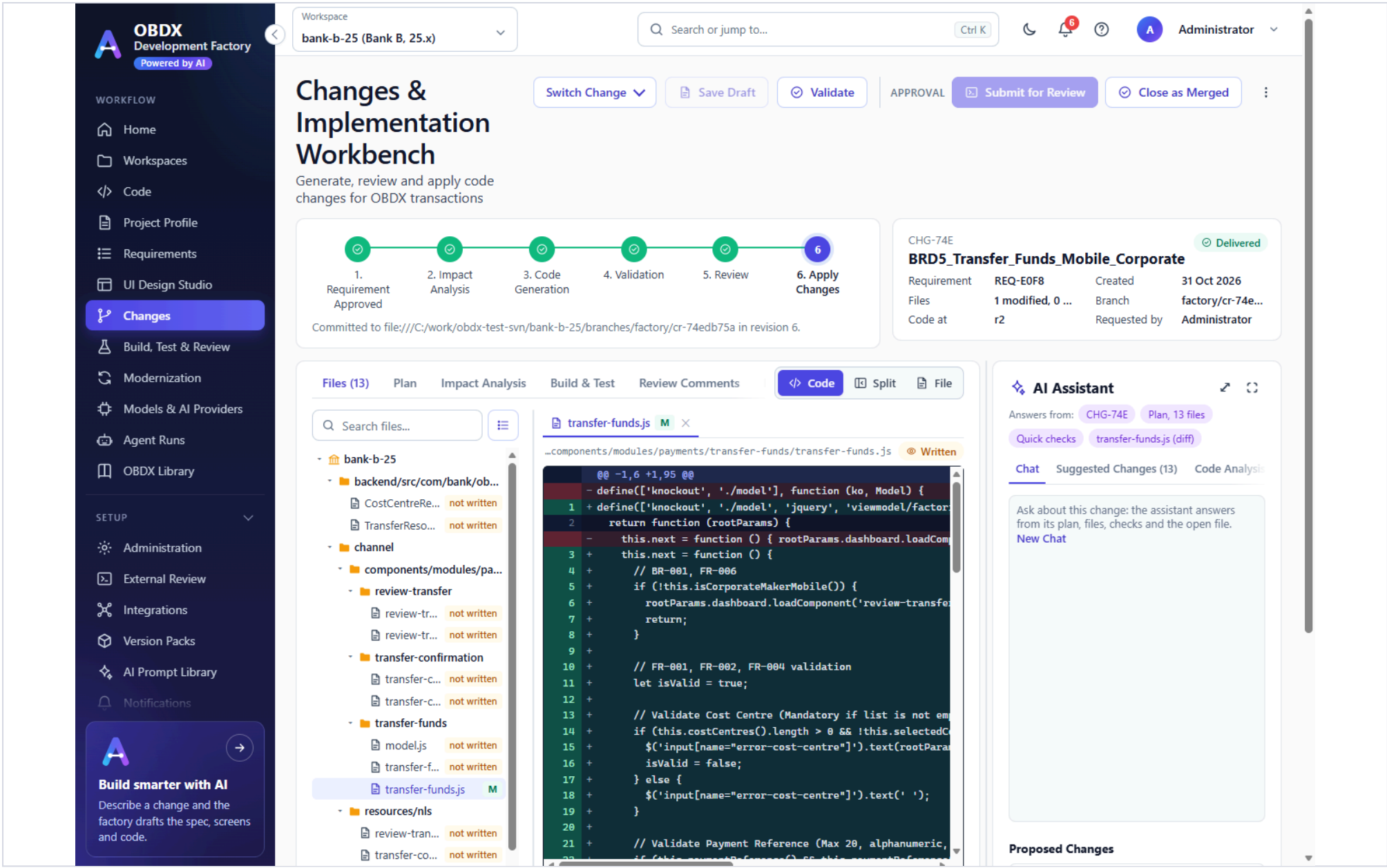


Visual Designer. One screen in detail: OBDX components, fields with validations and conditions, actions, channel previews, and checks against the requirement.

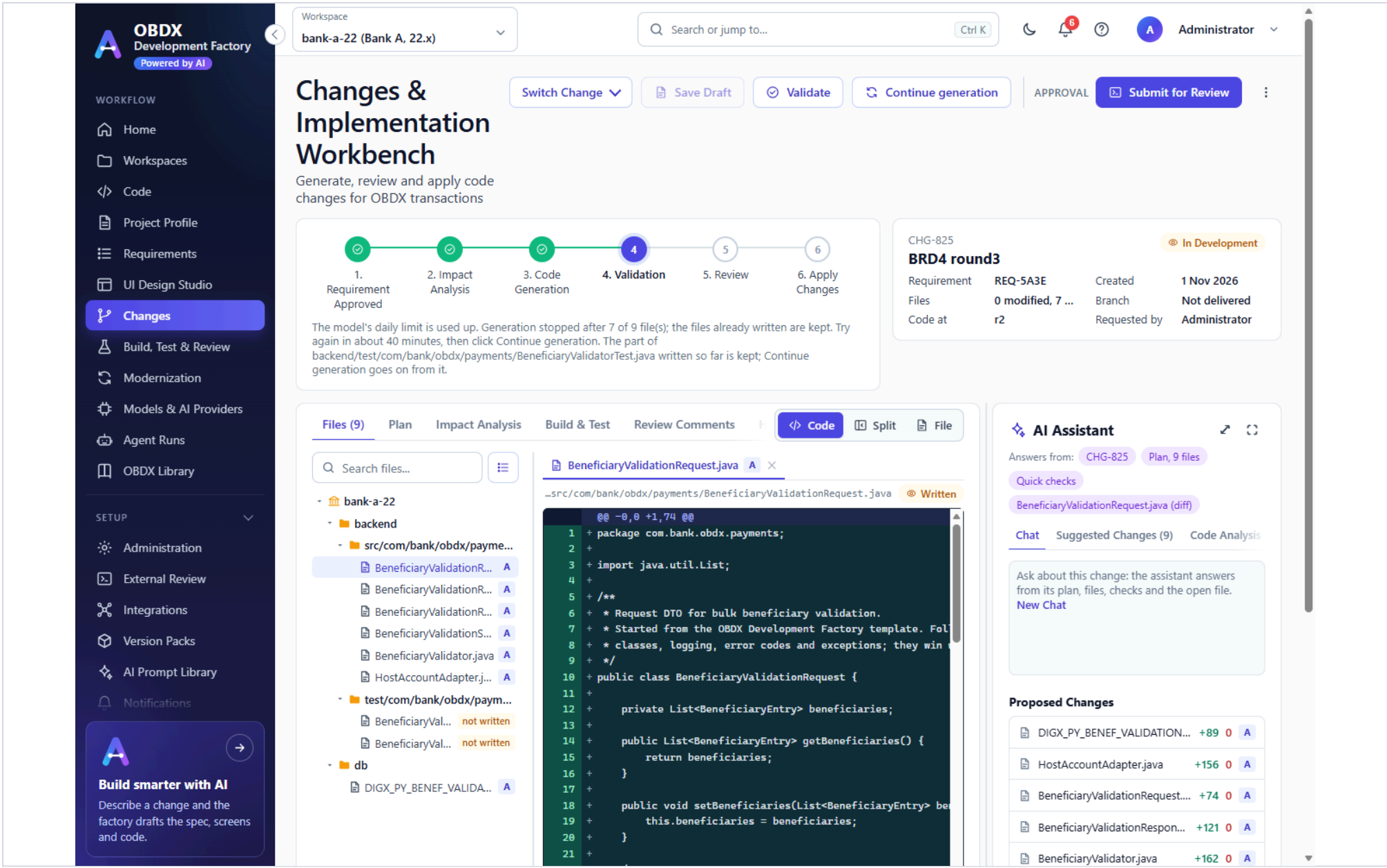
Impact analysis and the Changes workbench



Impact Analysis. Every file to create or modify (UI, resource bundles, REST resource, service, DTOs, SQL, tests) in dependency order, with risks such as a missing test, an API no file serves, or a new class on one of OBDX's own paths.

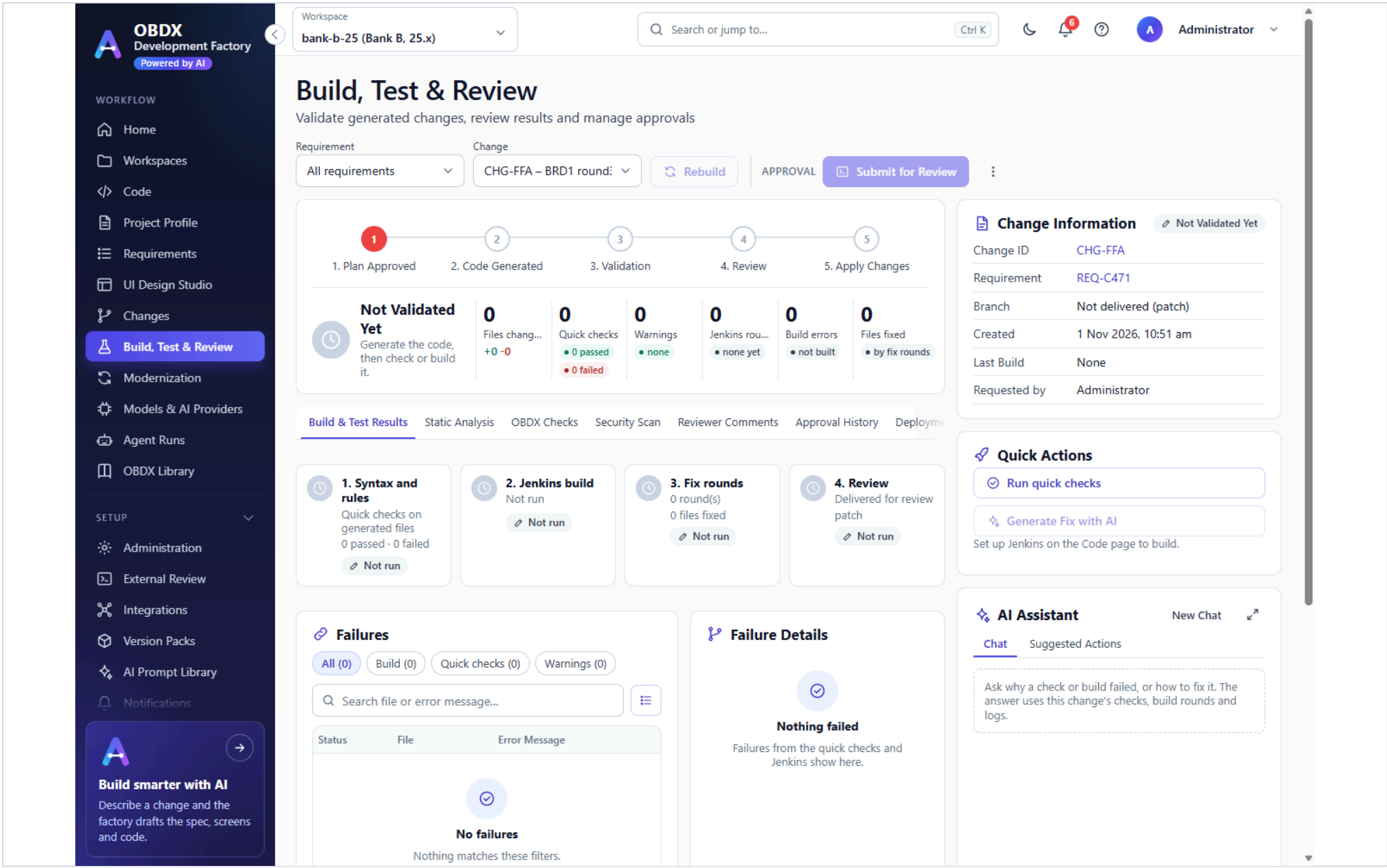


Changes workbench, BRD5 (customise an existing transaction). Ten existing files modified and three created, gated to mobile corporate makers, delivered to an SVN branch. The diff shows what changed; the AI assistant answers questions about the change.

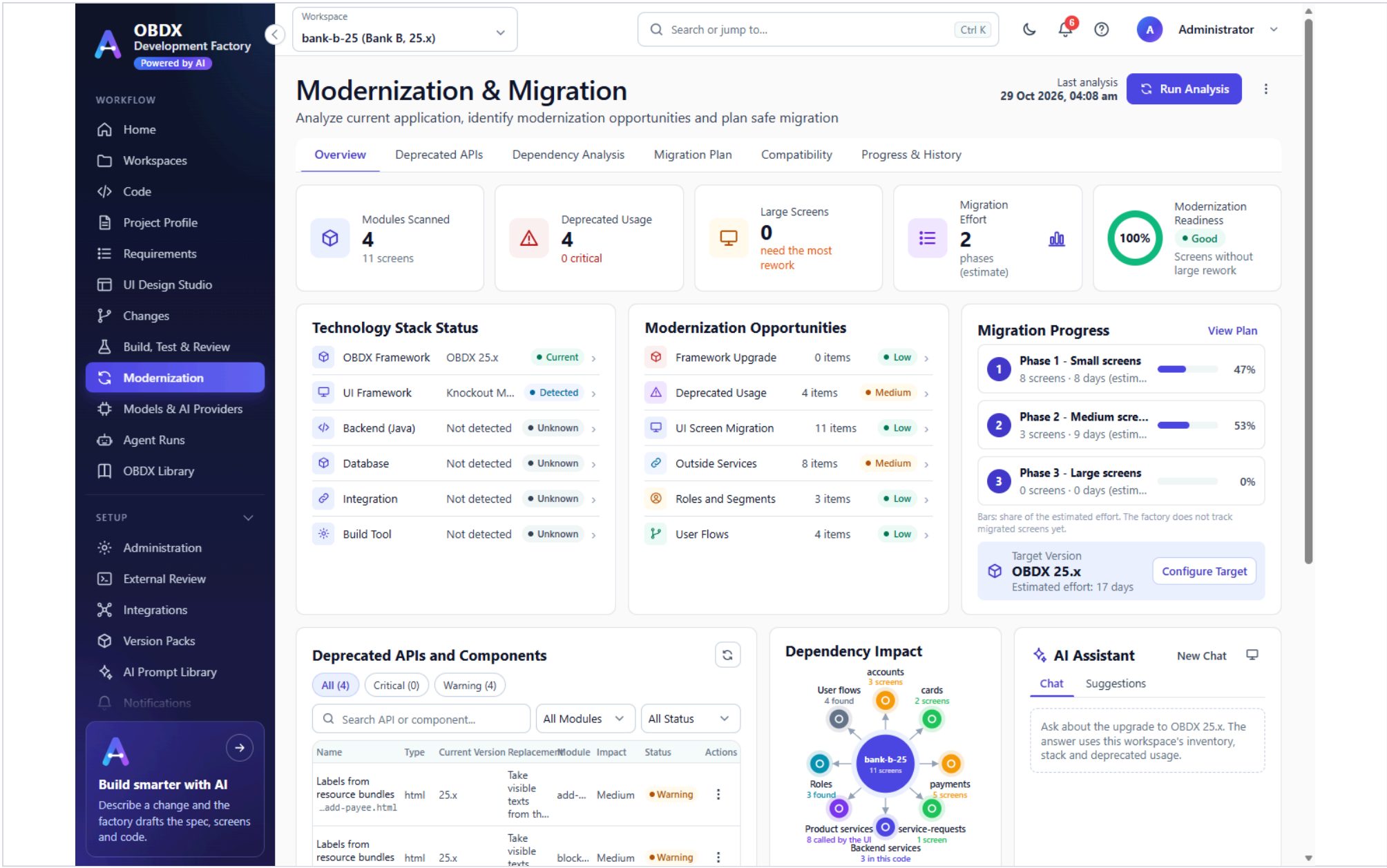


Changes workbench, BRD4 (backend REST API). A change stopped by the model's daily limit: 7 of 9 files written, the rest marked not written, a cut-off file kept, and Continue generation for when the limit resets.

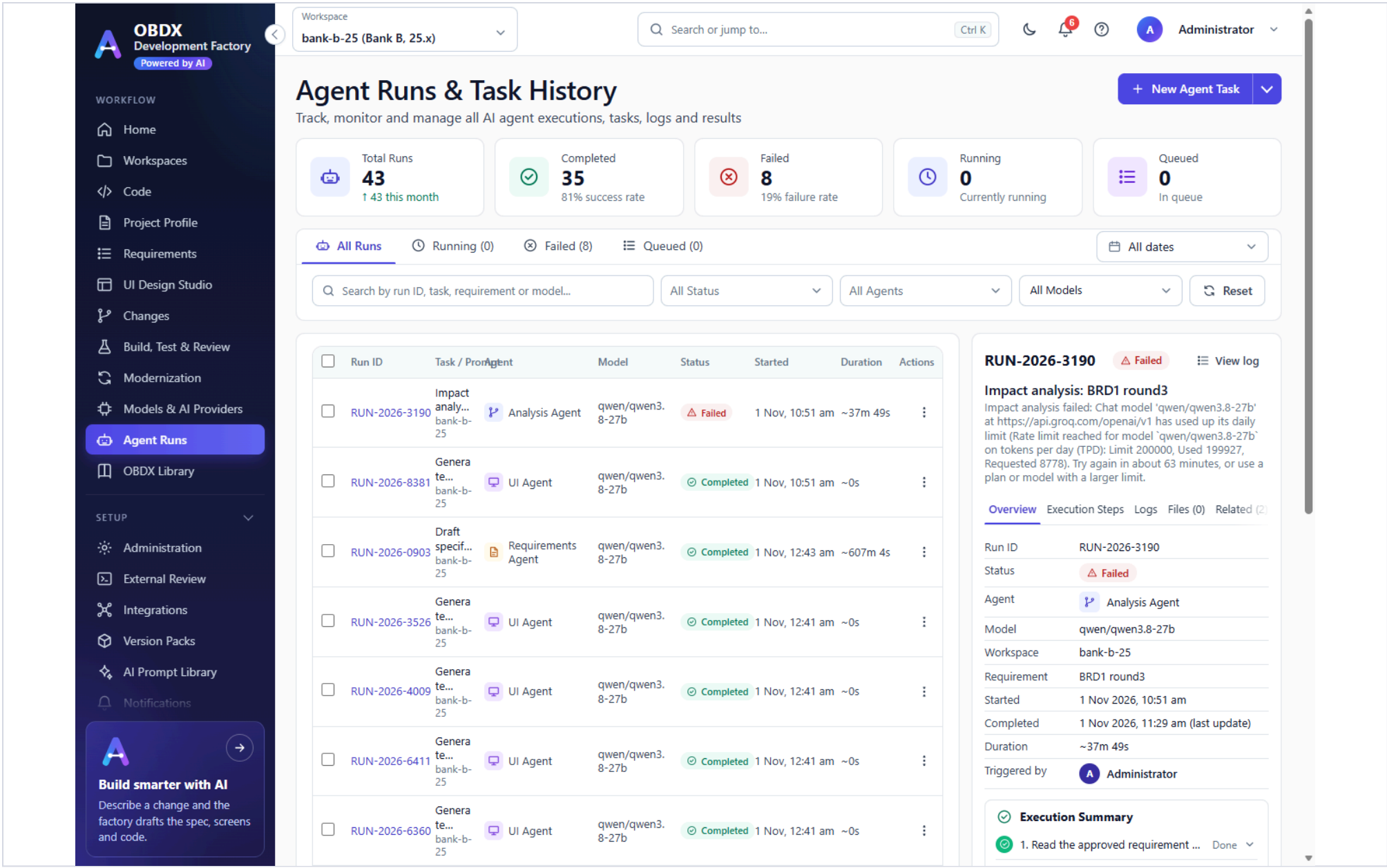
Build, test and review; modernization; agent runs



Build, Test & Review. Jenkins builds of each change, test results, review comments and the checks run on the generated code.

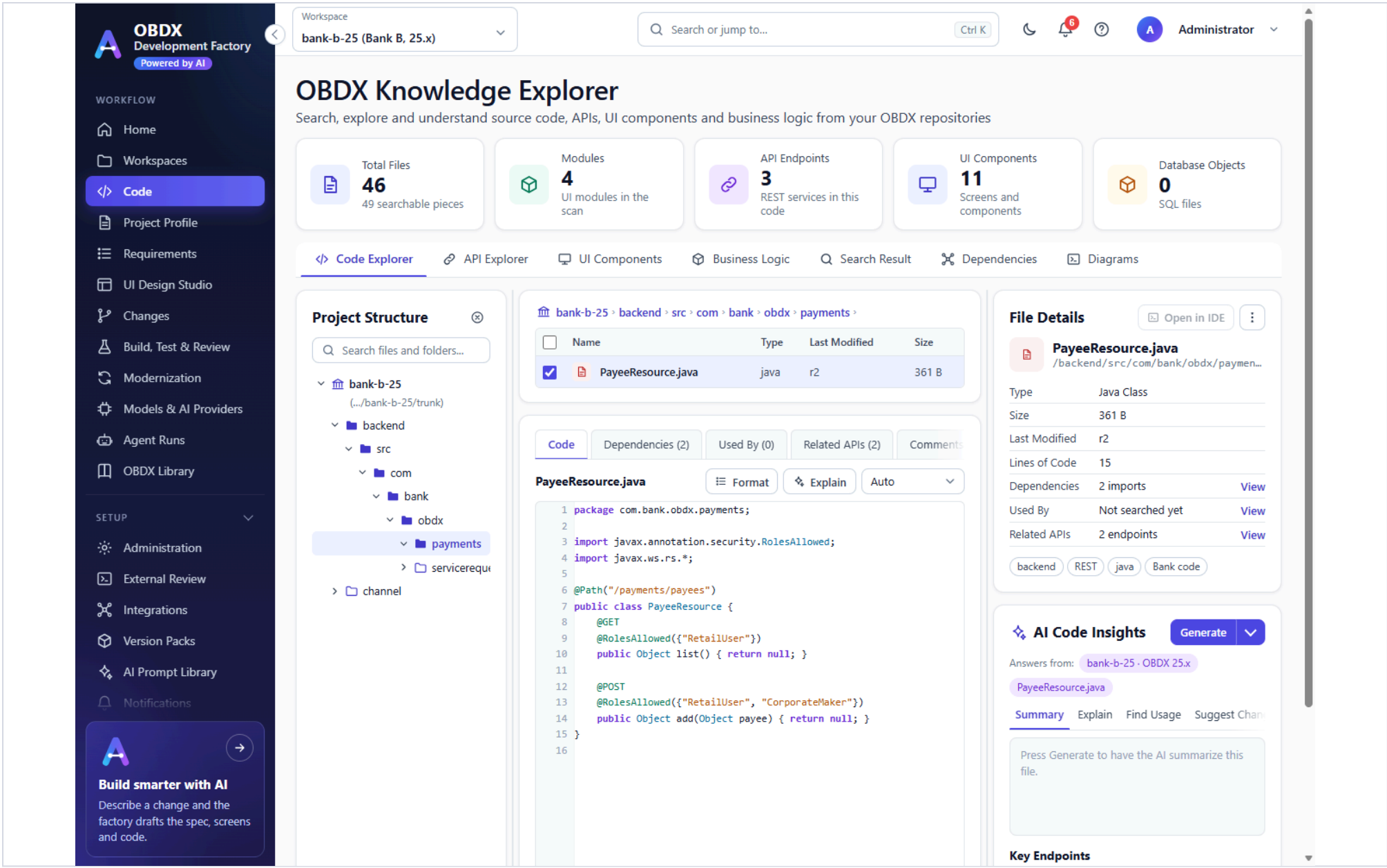


Modernization. Plans for moving screens from one OBDX version or channel to another (for example JET to React Native).

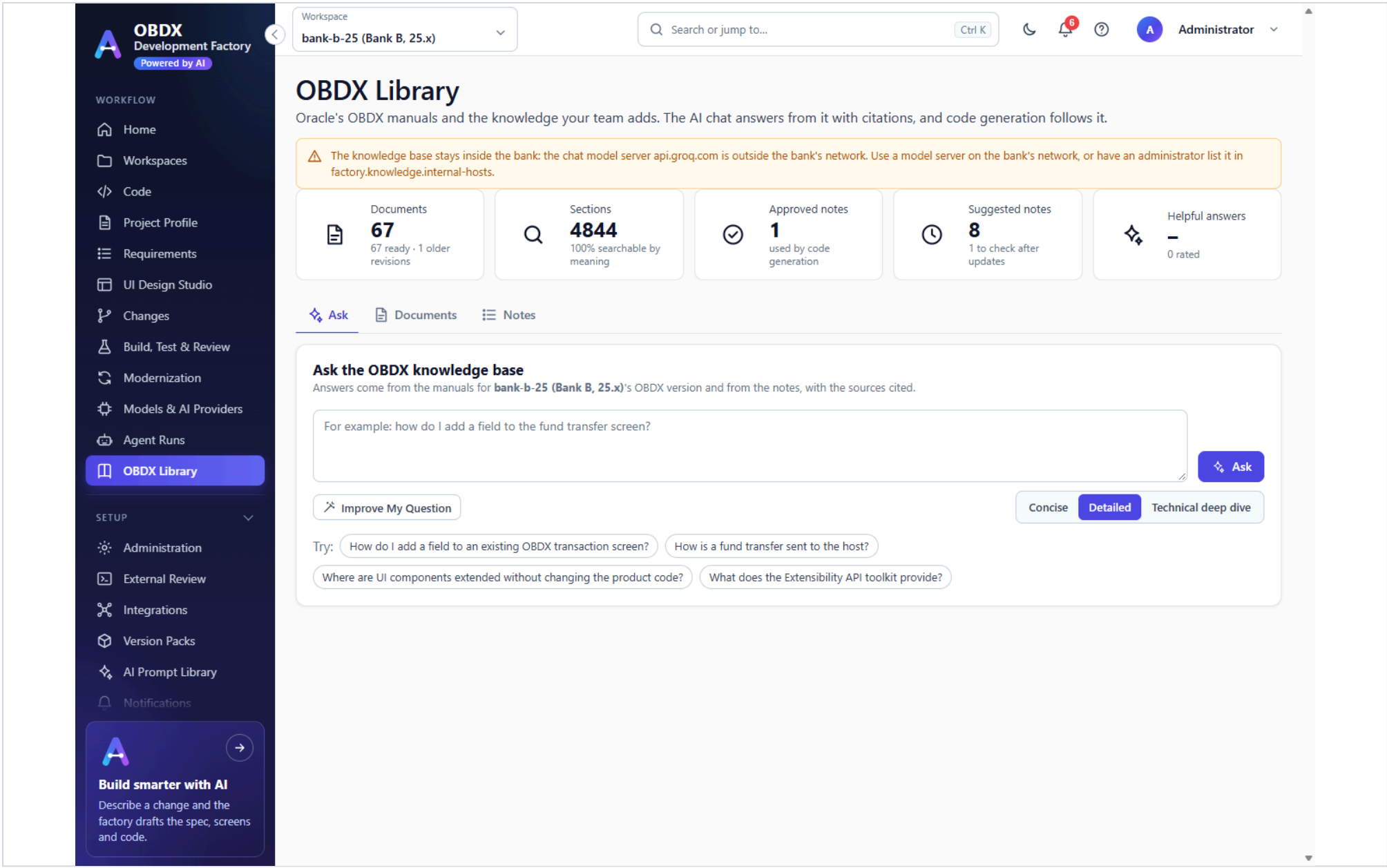


Agent Runs. Every AI run (drafting, design, planning, generation) with its model, timing and outcome.

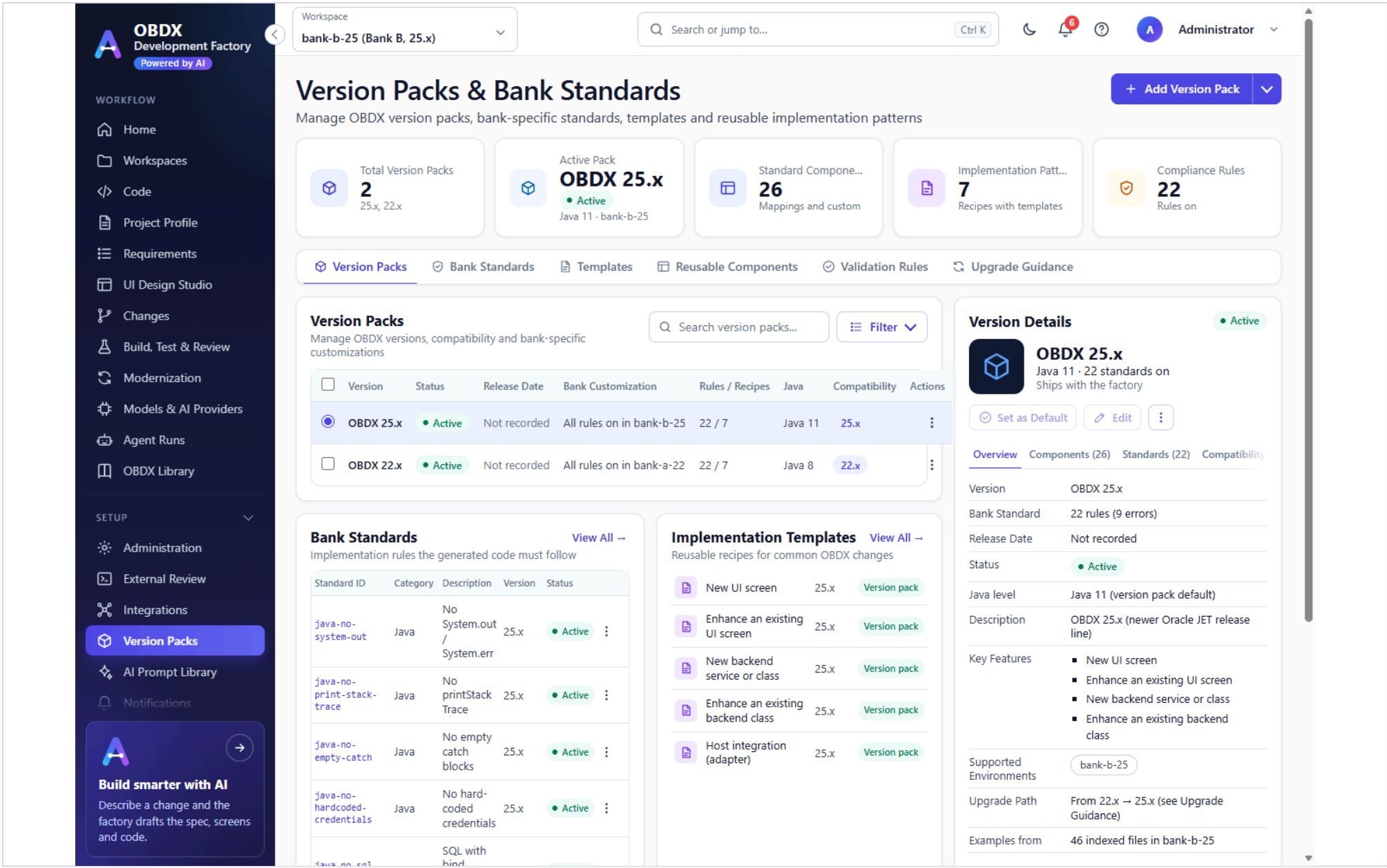
Knowledge



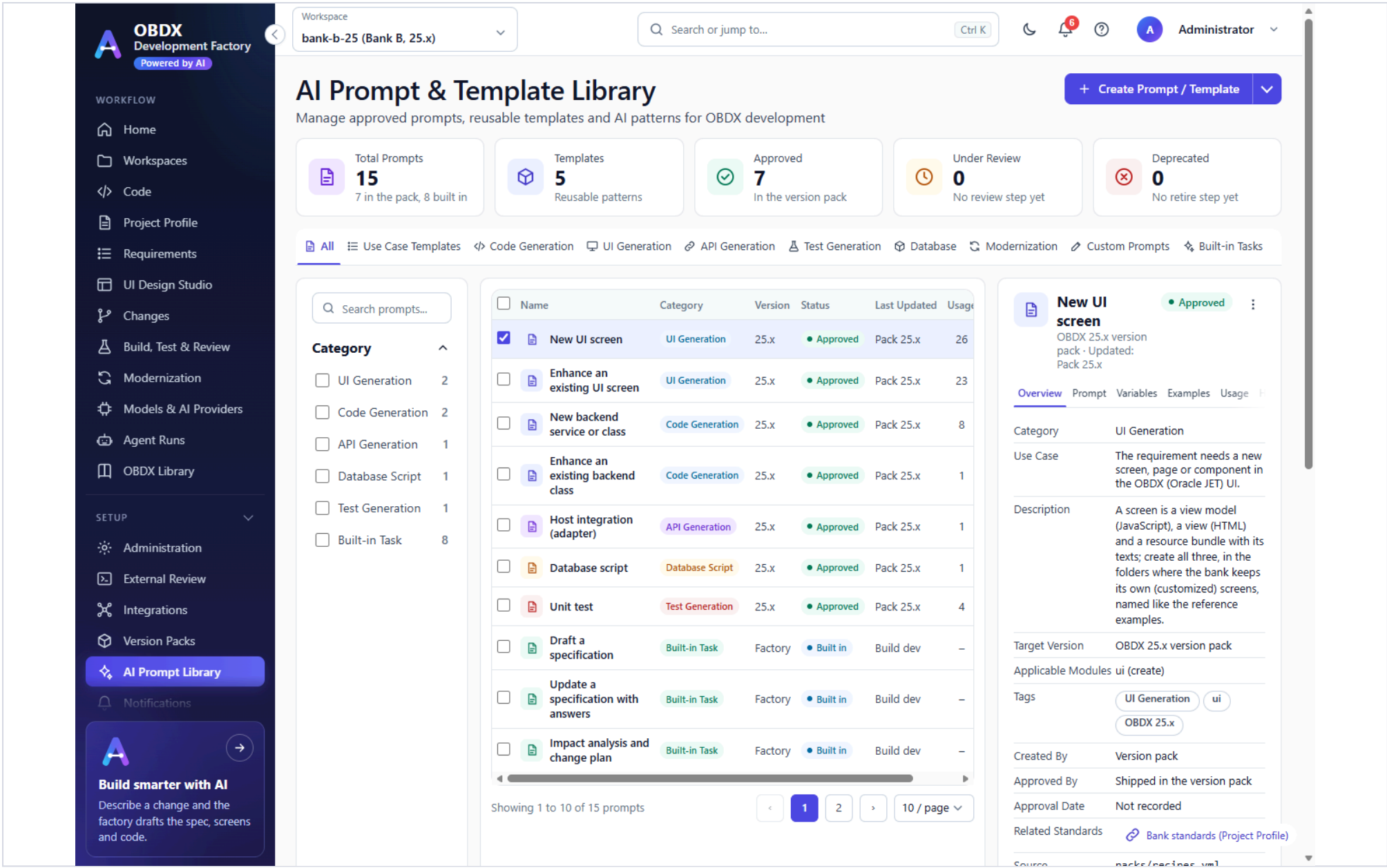
Knowledge Explorer. Questions across the bank's code, documents and the OBDX library; answers cite their sources.



OBDX Library. Oracle's OBDX documentation and the bank's own documents and notes, imported and searchable.

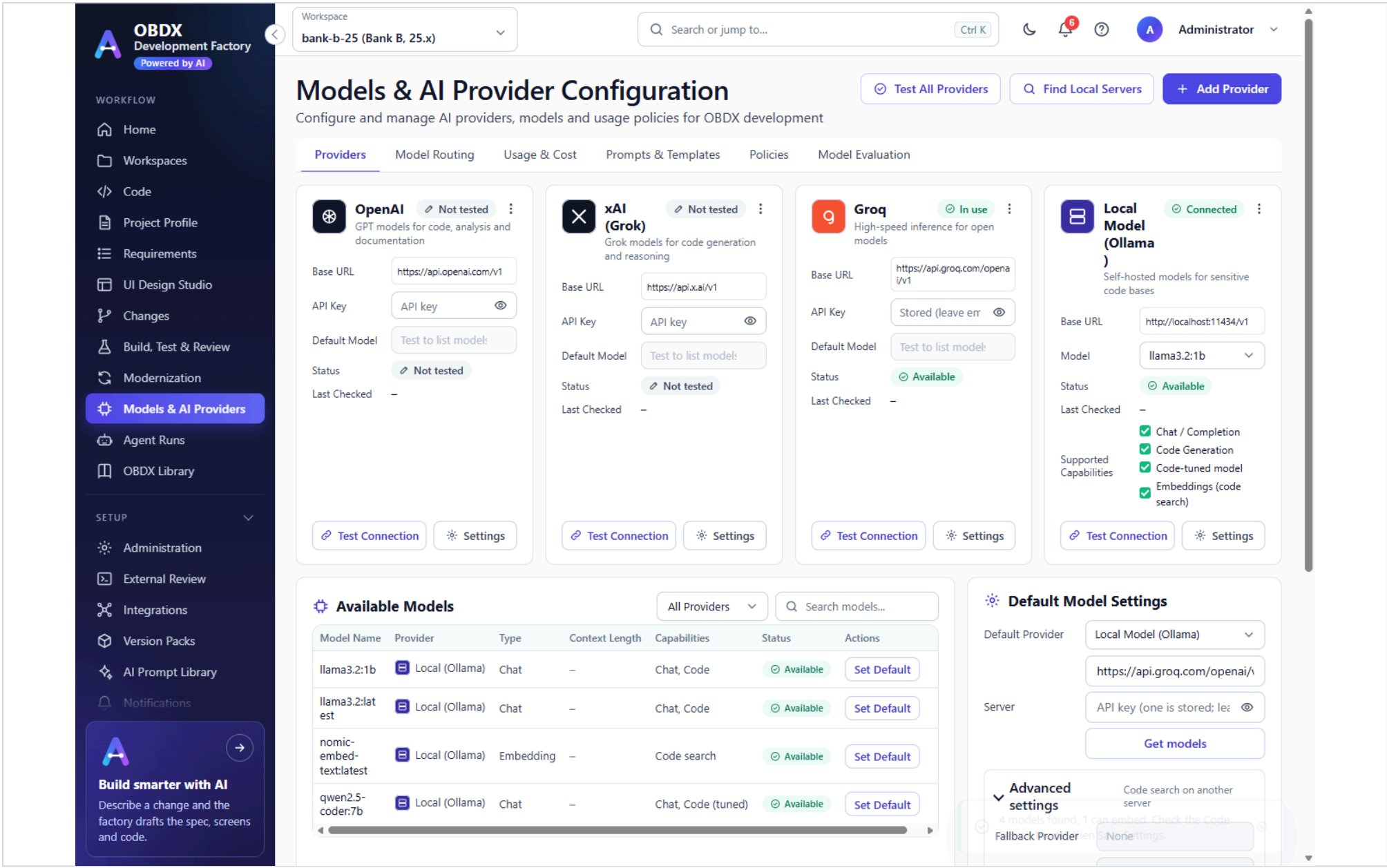


Version Packs. Rules, templates and checks per OBDX version and target (JET, React, React Native) that generated code must follow.

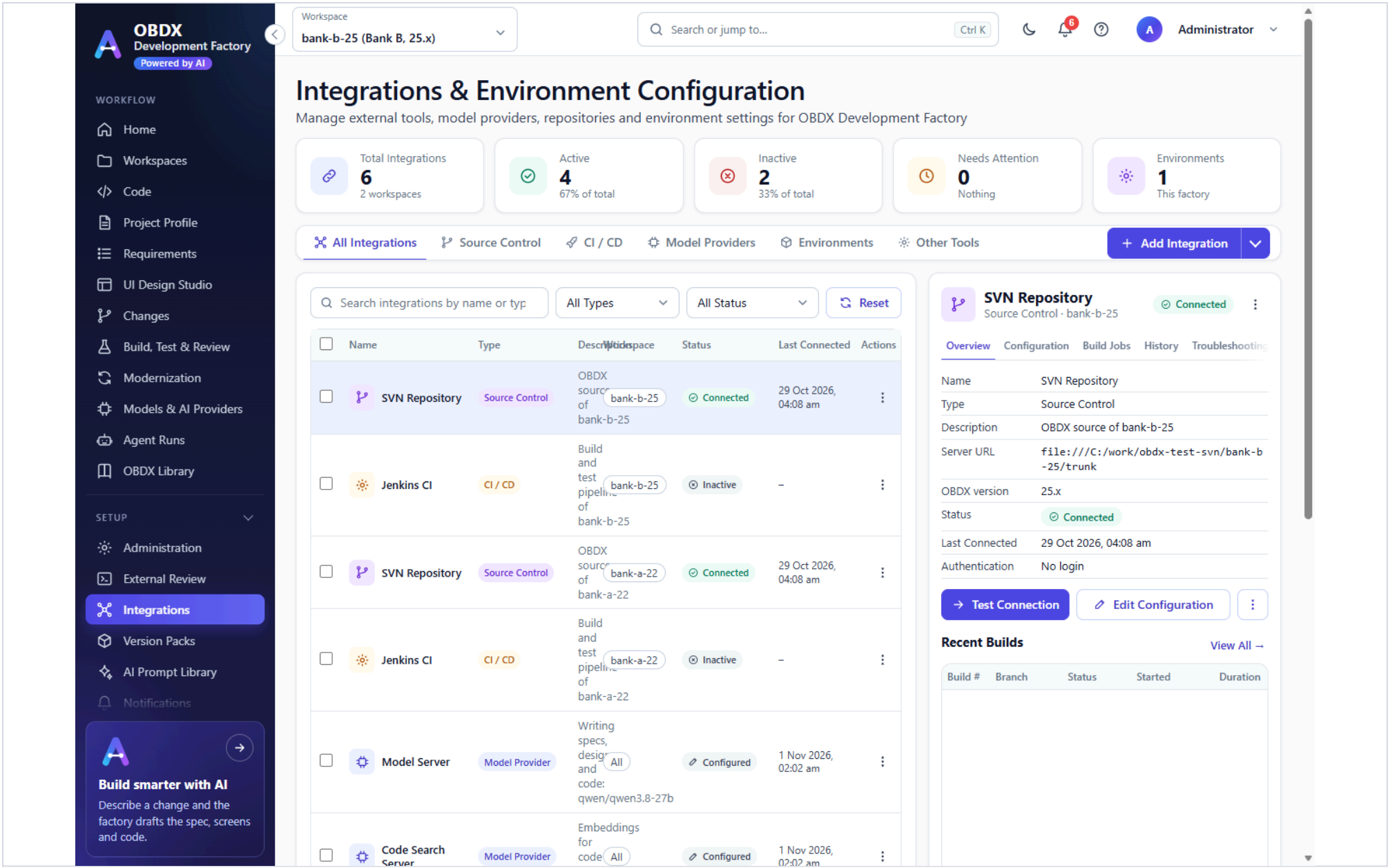


AI Prompt Library. The prompts the factory uses, which administrators can adjust.

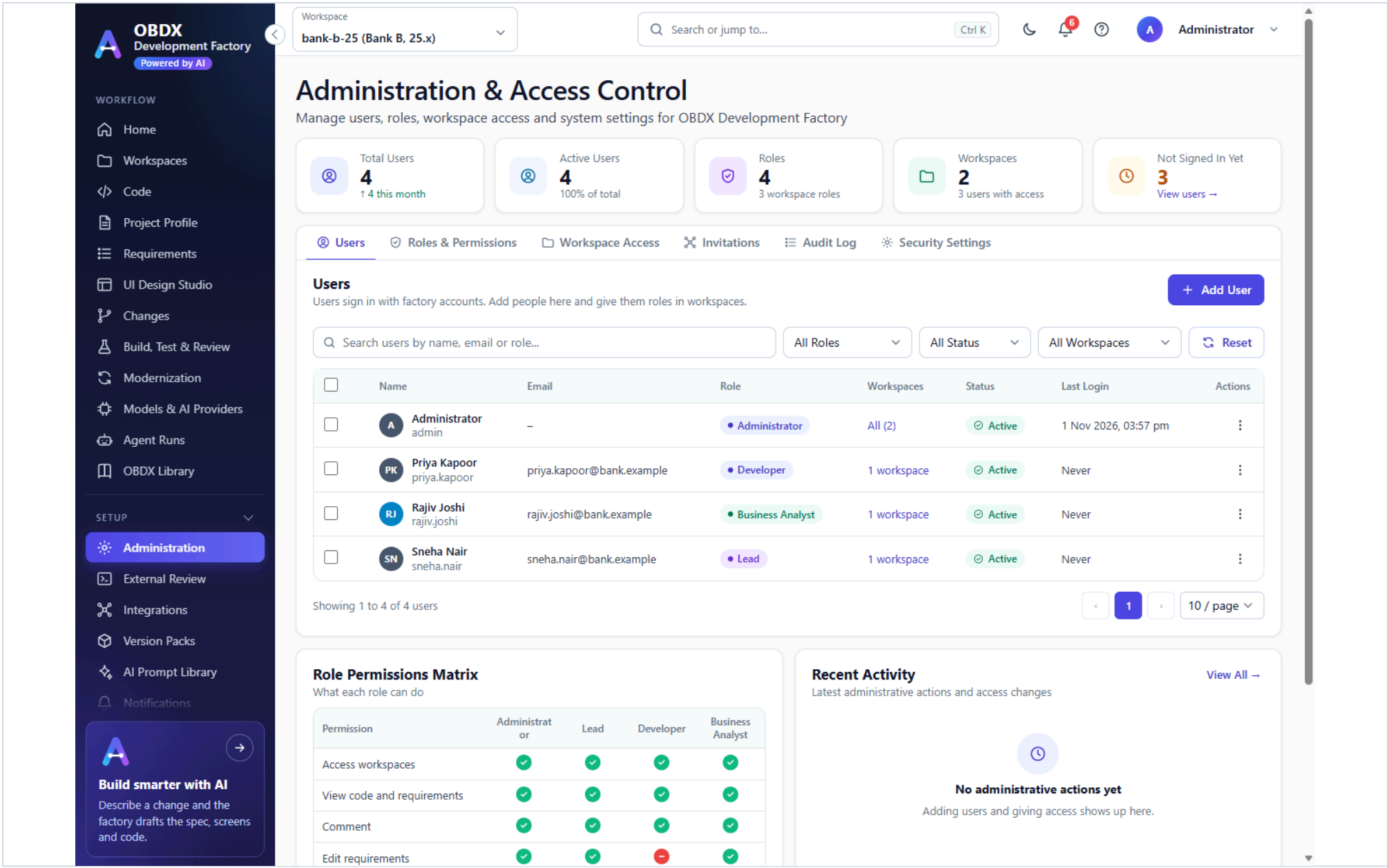
Setup



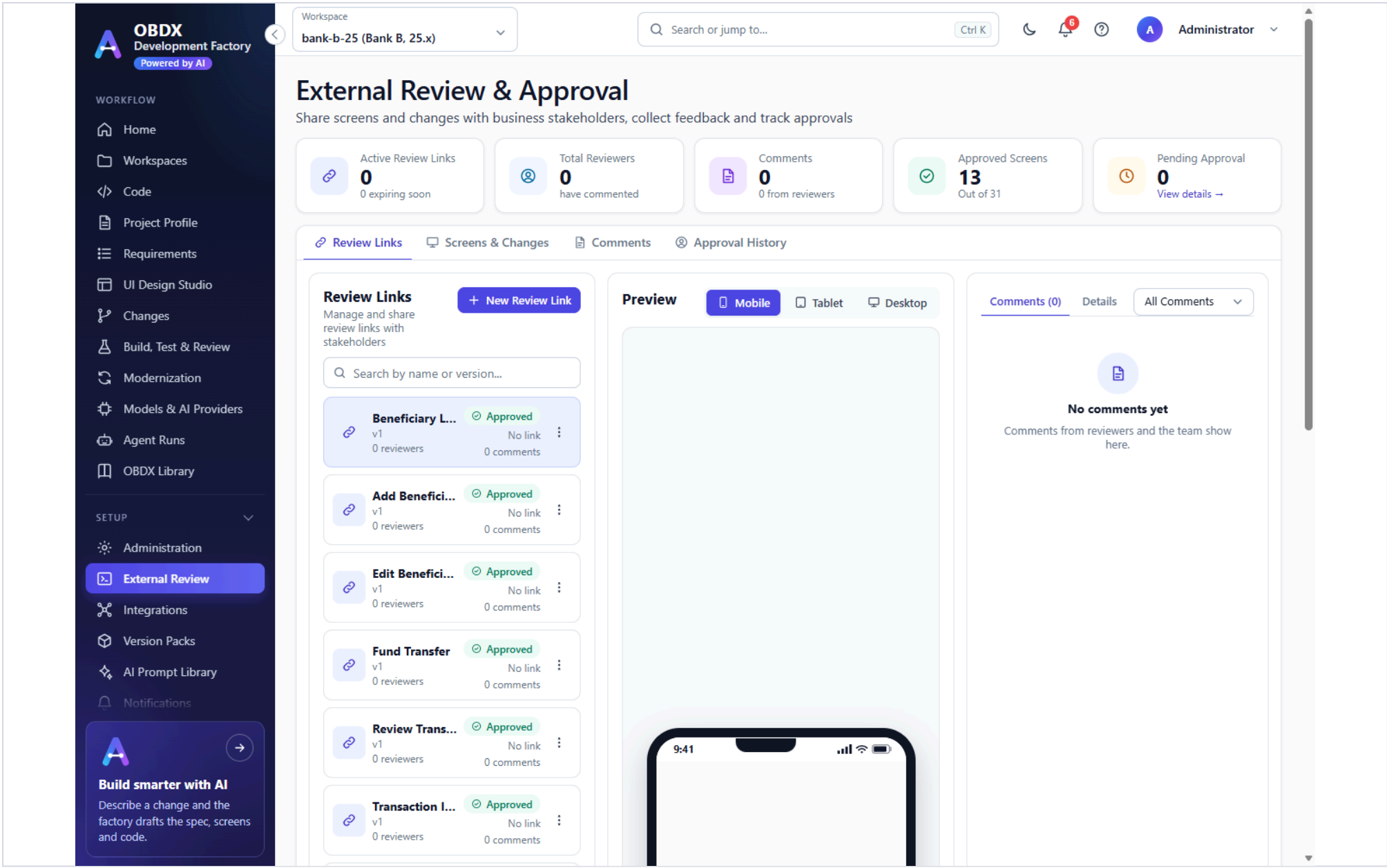
Models & AI Providers. Local models (Ollama) and hosted ones (Groq, OpenAI, xAI and any OpenAI-compatible server), which model does what, and policies such as keeping bank knowledge away from external models. Keys are stored encrypted and never shown.



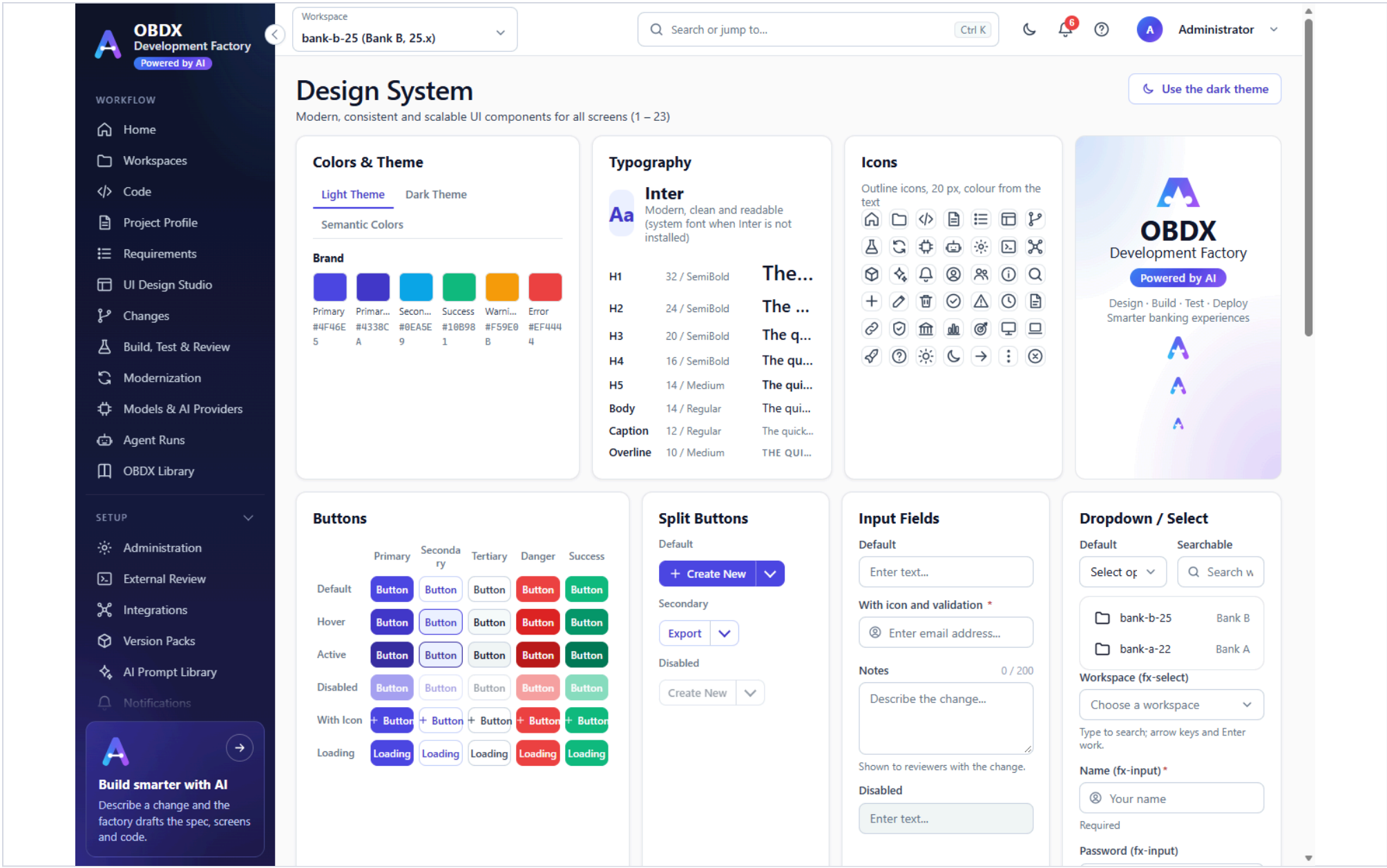
Integrations. SVN, Jenkins and the bank's sign-in (LDAP, single sign-on).



Administration. Users, roles and per-workspace permissions, audit log, updates.



External Review. Send a change to reviewers outside the factory and collect their comments.



Design System. The factory's own components and colours, light and dark.

Known gaps and questions for reviewers

- Reading BRDs, drafting specs, designing screens, approval and delivery work for all five kinds of change.
- Generated code is not yet production-ready: with a small free model it still has compile errors between files and gaps against the BRD's rules. The factory now catches most of these and sends them back to the model.
- A free hosted model's daily limit stops generation part-way; the factory keeps what was written and continues later.
- The screen designs use placeholder sample values and sometimes generic options (Option 1, Option 2) where the BRD does not name them.

1. Which screens or steps could be merged or removed to make the flow from BRD to code shorter?
2. Is the UI Design Studio clear enough for a business analyst, and what would make the phone and desktop previews more convincing to a bank?
3. What should a developer see first on the Changes workbench?
4. Which features are missing that a bank's OBDX team would expect (for example test data, API mocks, release notes, approvals by role)?
5. Which pages feel cluttered, and what would you simplify on each?